

Documento de Arquitectura
4+1 View Model

Integrantes

Francisco Infanti
110822

Franco Ariel Alani
111147

Lautaro Torraca
108813

1. Introducción 4
1.1 Propósito 4
1.2 Alcance 4
1.3 Casos de Uso 5
1.4 Decisiones de Arquitectura 5
1.4.1 Procesamiento por etapas 5
1.4.2 Punto único de entrada (SPoE) 6
1.4.3 Identificación de clientes 6
1.4.4 Comunicación entre Cliente y Gateway 6
2. Logical View 7
2.1 Diagrama DAG 7
Client 7
Gateway 8
Middleware 8
Filter 8
Image Downloader 8
Word Reader 9
Sum Count 9
Join Top 9
Sum Amount 9
Average 9
Join Average 9
2.2 Diagrama de Robustez 10
2.2.1 Job 1 11
2.2.2 Job 2 12
2.2.3 Job 3 13
2.2.4 Job 4 15
2.2.4 Job 5 16
3. Process View 17
3.1 Procesos concurrentes 17
3.2 Query 1 18
Diagrama de Secuencia 18
Diagrama de actividad 19
3.3 Query 2 20
Diagrama de Secuencia 20
Diagrama de actividad 21
3.4 Query 3 22
Diagrama de Secuencia 22
Diagrama de actividad 24
3.5 Query 4 25
Diagrama de Secuencia 25
Diagrama de actividad 26

3.6 Query 5 27
Diagrama de Secuencia 27
Diagrama de actividad 28
4. Development View 28
4.1 Diagrama de Paquetes 28
5. Physical View 29
5.1 Diagrama de Despliegue 29
6. Anexo 30
6.1 Listado de Tareas **30**

1. Introducción

1.1 Propósito

El presente documento describe la arquitectura del sistema distribuido **Wikipedia Article Analysis**, utilizando el modelo 4+1 Views: se presenta la arquitectura desde las vistas lógica, de procesos, de desarrollo y física, complementadas por escenarios de ejecución que permiten relacionar dichas vistas

1.2 Alcance

El sistema distribuido **Wikipedia Article Analysis** tiene por objetivo procesar un conjunto de artículos de Wikipedia, escrito en los idiomas inglés y francés, para resolver cinco consultas de análisis sobre sus metadatos (idioma, abstracto, fecha de modificación, URL, etc.).

El cliente realizará una subida de los artículos de Wikipedia sobre los cuales se quiera realizar el análisis, y el sistema deberá poder resolver en paralelo las siguientes cinco consultas:

1. Obtener la URL y la fecha de modificación de los artículos escritos en inglés cuya fecha de modificación sea hasta 2020 inclusive.
2. Obtener nombre, URL e imagen de los artículos escritos hasta 2020 inclusive que contengan una sección denominada biography o biographie.

   La descarga de imágenes requerida se integra con un servicio externo basado en la API de Wikimedia, tal como sugiere el enunciado.

3. Obtener el conjunto de palabras compartidas entre artículos escritos en inglés y artículos escritos en francés.
4. Obtener las veinte palabras más utilizadas entre todos los abstracts, excluyendo stopwords y contabilizando cada palabra una única vez dentro de cada abstract
5. Obtener la URL y cantidad de palabras del abstract de los artículos escritos a partir de 2021 cuya cantidad de palabras sea superior al promedio correspondiente a los artículos escritos hasta 2020 inclusive.

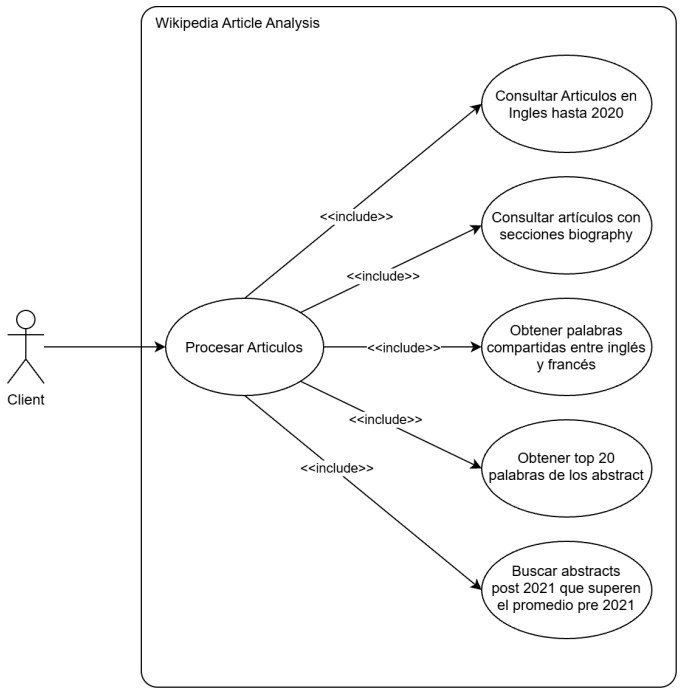
La arquitectura diseñada debe permitir atender a múltiples clientes de forma concurrente, manteniendo aisladas sus ejecuciones; no se requiere soportar más de una ejecución simultánea por cada cliente. Asimismo, el sistema debe estar preparado para entornos multicomputadora y permitir incrementar la cantidad de elementos de cómputo para adaptarse tanto al aumento en la cantidad de clientes, como al volumen de datos.

La comunicación interna entre los elementos del sistema se realiza mediante un middleware orientado a grupos; este middleware abstrae a los componentes de la infraestructura concreta utilizada para el intercambio de mensajes, que en nuestro caso es

RabbitMQ. A su vez, el diseño contempla la finalización ordenada de los procesos frente a señal de SIGTERM, de acuerdo con el requerimiento de graceful quit.

1.3 Casos de Uso

A continuación se presenta un diagrama de caso, donde se exponen las funcionalidades principales del sistema desde el punto de vista del cliente; para cada ejecución, el cliente envía un conjunto de artículos de Wikipedia y el sistema los procesa para resolver las cinco consultas en conjunto.



1.4 Decisiones de Arquitectura

1.4.1 Procesamiento por etapas

El procesamiento de los datos se divide en componentes especializados, donde cada uno es responsable de una operación concreta. Por lo que aquellas consultas que requieren de múltiples transformaciones, se organizan como un pipeline de procesamiento compuestos por varias etapas que involucran varios componentes.

Esta separación en etapas permite distribuir los componentes entre distintas unidades de cómputo, permitiendo escalar de forma independiente aquellos componentes que presenten una mayor carga.

1.4.2 Punto único de entrada (SPoE)

El servidor del **gateway** actúa como un único punto de entrada al sistema, de esta forma los clientes se abstraen de toda la complejidad del sistema distribuido, y solamente se comunican con un único proceso; sus responsabilidades son:

- Recibir los artículos enviados por el cliente.
- Distribuir la información de los artículos hacia el sistema.
- Recibir los resultados producidos por cada una de las cinco consultas.
- Devolver el resultado al cliente.

El gateway no implementa ningún tipo de lógica de análisis de los artículos recibidos, simplemente actúa como un intermediario entre el sistema y el cliente.

1.4.3 Identificación de clientes

Cada ejecución de un cliente se identifica mediante un **req_id**, un contador incremental único que maneja el gateway, y asigna a cada cliente cuando el mismo se conecta a este.

Este identificador acompaña a todos y cada uno de los mensajes que envía el sistema en las distintas etapas de procesamiento, permitiendo distinguir la información perteneciente a ejecuciones de clientes concurrentes; de esta manera, diferentes clientes pueden compartir los mismos recursos de procesamiento sin mezclar sus resultados.

Por su parte, el cliente utiliza un **client_id** para organizar los archivos de salidas locales, así los resultados se almacenan en la carpeta de *output* ordenados por **client_id**.

1.4.4 Comunicación entre Cliente y Gateway

El cliente y el gateway se comunican mediante una única conexión TCP/IP bidireccional, que permite enviar artículos y recibir resultados simultáneamente.

A su vez, el cliente envía los artículos completos agrupados en lotes mediante mensajes de DATA. Al terminar el envío, transmite un mensaje de EOF y mantiene la conexión abierta para continuar recibiendo resultados.

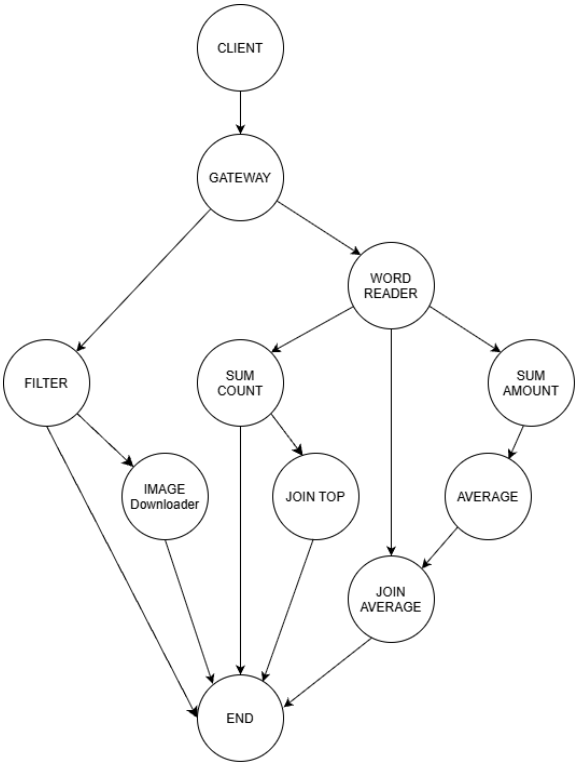
El gateway devuelve los resultados mediante mensajes de DATA, indicando a qué consulta pertenece cada uno. Y cuando finalizan las cinco consultas, envía un único mensaje de EOF global y cierra la conexión.

Los mensajes incluyen una cabecera que indica el tipo de mensaje, la cantidad de registros y la longitud del contenido, permitiendo interpretar correctamente los datos transmitidos.

2. Logical View

2.1 Diagrama DAG

El **diagrama de DAG (Directed Acyclic Graph)** nos proporciona una vista general de los componentes principales del sistema que se van a utilizar para resolver todas las consultas planificadas para los artículos que envía el cliente.



Client

Representa el punto de origen de una ejecución, su responsabilidad principal es obtener, leer y normalizar los artículos del dataset de Wikipedia, transmitirlos al sistema a través del Gateway y almacenar los resultados recibidos.

Su función se limita principalmente a:

- Obtener los archivos Parquet.
- Leer los artículos por lotes y normalizar sus campos al formato esperado por el sistema.
- Establecer una conexión TCP con el Gateway para iniciar una ejecución.
- Enviar los artículos completos agrupados en lotes y señalar la finalización del envío mediante un mensaje de EOF.
- Recibir los resultados de las cinco consultas y guardarlos en archivos CSV independientes.
- Guardar las imágenes recibidas como parte de los resultados de la consulta 2.

El cliente ejecuta dos hilos: uno dedicado a la lectura y el envío de artículos, y otro a la recepción y el almacenamiento de resultados, permitiendo realizar ambas tareas de forma concurrente.

Los archivos de resultados se mantienen con el sufijo *.partial* hasta completar correctamente el envío y recibir el mensaje de EOF global del gateway, que indica la finalización de las cinco consultas, en dicho caso se renombran con la extensión *.csv*.

Gateway

Constituye el punto de entrada al sistema distribuido y actúa como intermediario entre los clientes y los componentes internos de procesamiento.

Sus principales responsabilidades son:

- Recibir los artículos enviados por los clientes.
- Distribuir los datos hacia las distintas ramas de procesamiento.
- Iniciar concurrentemente las cinco consultas.
- Recibir los resultados finales generados por cada Job.

Middleware

Proporciona una abstracción común para la comunicación entre los procesos distribuidos del sistema. Su objetivo principal es evitar que los componentes de negocio dependan directamente de los mecanismos concretos utilizados para enviar y recibir mensajes.

Este abstrae la comunicación basada en grupos, permitiendo que un conjunto de procesos equivalentes pueda ser tratado como una única unidad lógica de procesamiento.

Entre sus responsabilidades conceptuales se encuentran:

- Envío y recepción de mensajes.
- Abstracción de queues y exchanges.
- Suscripción y desuscripción de consumidores.
- Administración del ciclo de vida de la comunicación.

Filter

Componente encargada de filtrar los artículos según su *date modified* (fecha de modificación), *in language* (lenguaje del artículo) *& sections* (secciones del artículo).

Image Downloader

Componente encargada de descargar la imagen de los artículos a través de la API de Wikipedia.

Word Reader

Componente encargada de realizar un análisis del *abstract* de los artículos: extrae y cuenta las palabras únicas (se ignoran las *stopwords*).

Sum Count

Componente encargado de contabilizar la cantidad total de apariciones de cada palabra única, por idioma y de forma total, de todos los artículos.

Join Top

Componente encargado de combinar los tops parciales de las palabras únicas con mayor cantidad de apariciones, con el objetivo de poder construir un único top final.

Sum Amount

Componente encargado de contabilizar la cantidad de palabras de un subconjunto de artículos (considerando *stopwords*), calculando una sumatoria parcial de cantidad de palabras de ese subconjunto de artículos.

Average

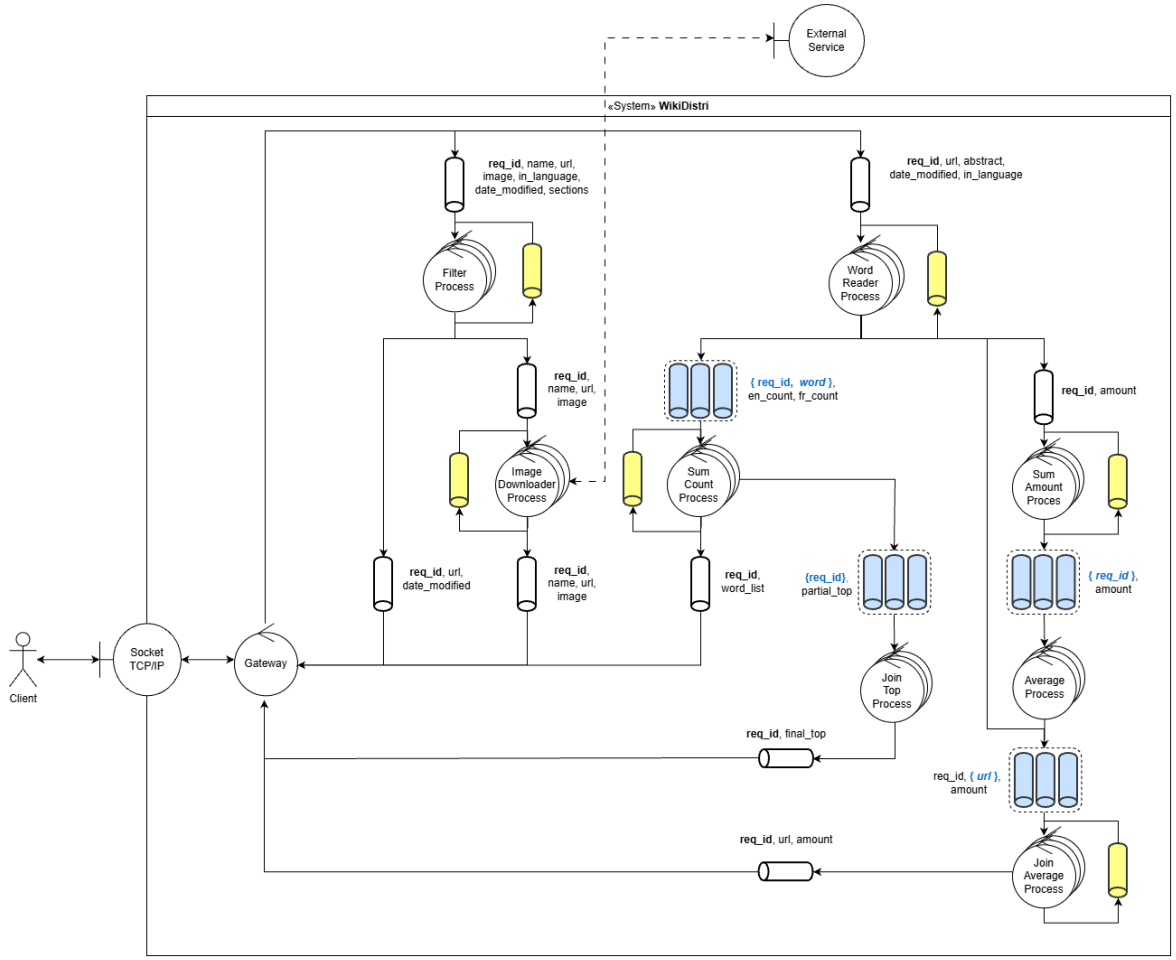
Componente encargado de calcular la sumatoria total de la cantidad de palabras del conjunto total de los artículos para poder determinar el promedio de palabras.

Join Average

Componente encargado de filtrar los artículos según si su promedio de palabras en el *abstract* es menor al promedio de palabras de un subconjunto selecto de artículos.

2.2 Diagrama de Robustez

El **diagrama de robustez** del sistema distribuido fue desarrollado para todo el sistema en general, y luego para cada consulta particular. De esta manera podemos observar como se ve todo el sistema distribuido para las cinco consultas, y los casos particulares para las consultas particulares.



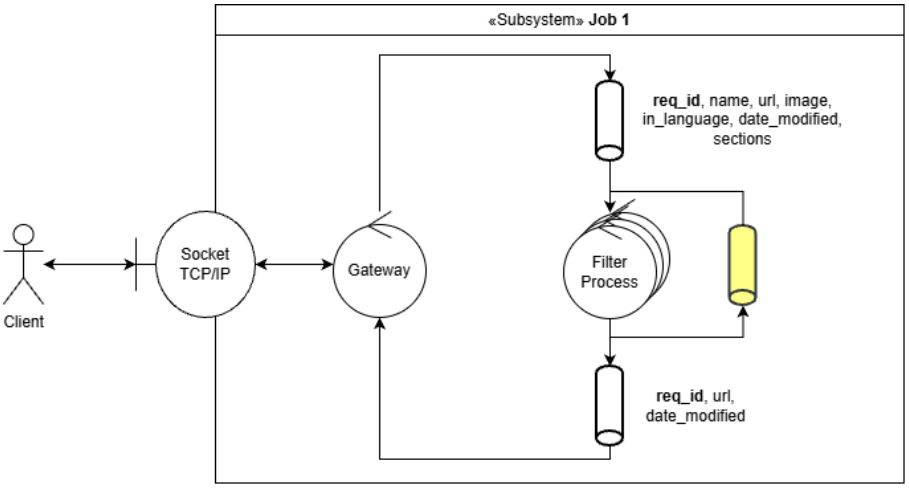
El cliente establece una única conexión TCP con el gateway, por medio de esta conexión envía de a lotes de artículos y recibe de forma concurrentemente los resultados de las consultas.

Como se mencionó anteriormente, el Gateway asigna un **req_id** a la conexión del cliente, y para cada artículo recibido genera las proyecciones de campos que necesitan los procesos de **filter** y **word reader**, publicando la información en sus respectivas *queues*.

Una vez consumida toda la entrada seleccionada, el cliente envía un único mensaje de EOF, el cual el gateway recibe e informa a los demás procesos del sistema, finalmente este devuelve un mensaje de EOF global al cliente cuando han finalizado las cinco consultas.

A continuación se exponen los diagramas de robustez para el caso particular de cada una de las consultas.

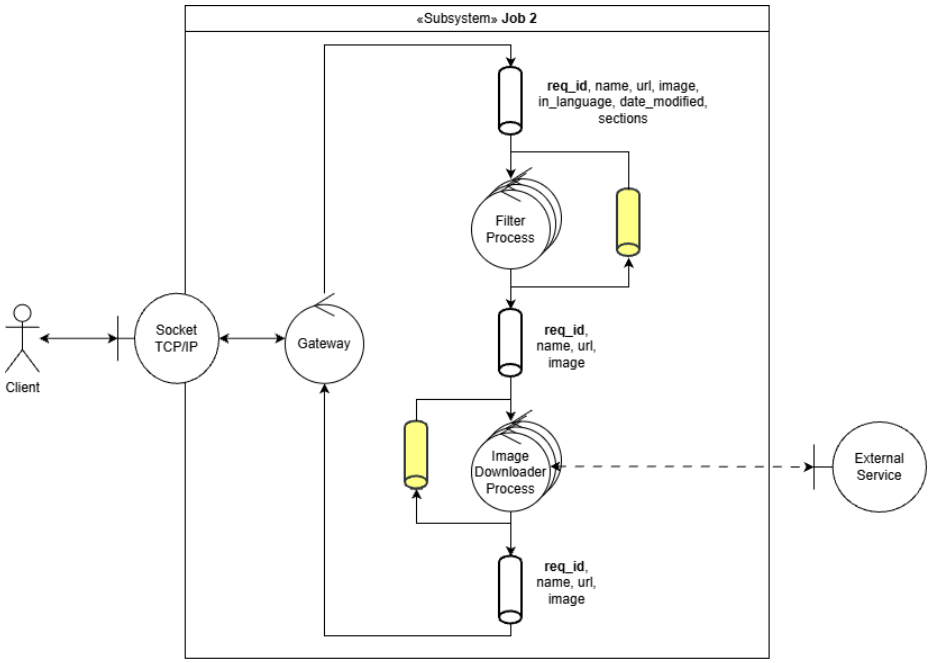
2.2.1 Job 1



El proceso de **filter** consume de la *queue* el *name*, *url*, *image*, *in_language*, *date_modified* y *sections* de un artículo particular, luego con esta información determina: si el articulo esta escrito en inglés y fue escrito antes o durante 2020, entonces satisface las condiciones de la primera consulta, y devuelve al gateway la *url* y el *date_modified*, publicando esa información en una *queue*.

Cuando el gateway no tiene más artículos que enviar, publica un mensaje de EOF en la *queue*, el cual será consumido por un único proceso de filter, por lo que es necesario que se coordinen de forma interna por medio de un *exchange* para que solamente el coordinador (el filter que consumió el EOF) publique un mensaje de EOF en la *queue* de la que consume el gateway para indicar que la primer consulta finalizó.

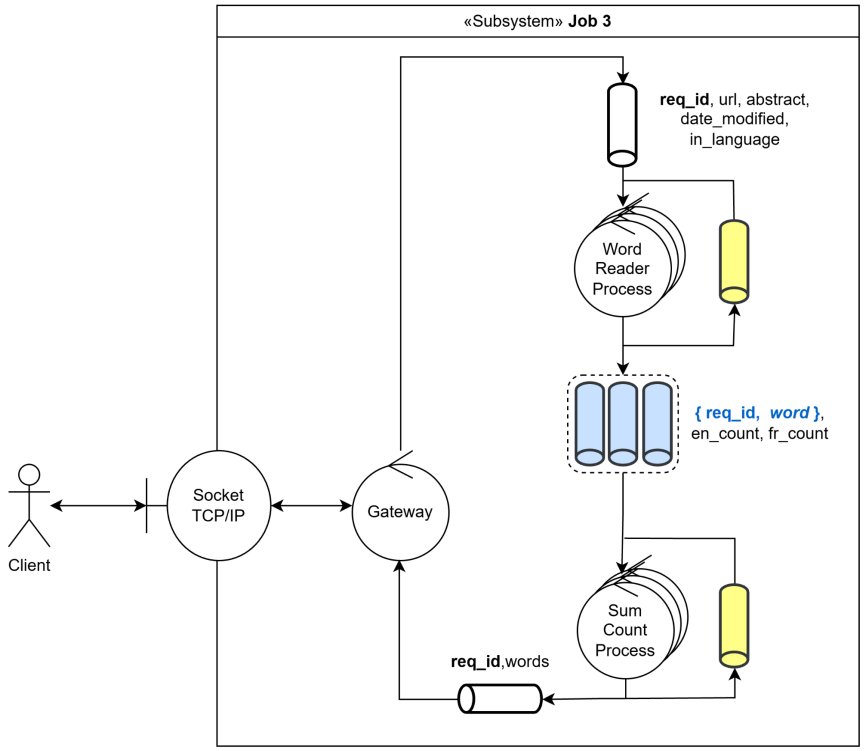
2.2.2 Job 2



Con la información que consume el proceso de filter de la *queue*, también determina: si el artículo fue escrito antes o durante 2020, y si contiene en su *sections* las palabras *biography* o *biographie*, entonces publica el *name*, *url* e *image* en otra *queue* de la que consume el proceso de **image downloader**; cuyo objetivo es interactuar con un **external service** para descargar la imagen. Una vez descargada, se publica el *name*, *url* y los *bytes* de la imagen en otra *queue* de la que consume el gateway para poder devolverlo al cliente.

Cuando el proceso de filter no tenga más artículos, simplemente envía un mensaje de EOF hacia uno de los image downloader (lo publica en la *queue* y uno lo consume), y estos de igual forma que antes se coordinan entre sí para que solamente uno envíe el mensaje de EOF hacia el gateway, indicando que la segunda consulta finalizó.

2.2.3 Job 3



El objetivo del proceso **word reader** es consumir de la *queue* los campos *url*, *abstract*, *in_language* y *date_modified* de cada artículo. A partir de la lectura de los *abstracts* recibidos, extrae y procesa la información necesaria para la resolución de la tercera, cuarta y quinta consulta.

Solo uno de los procesos word reader será responsable de consumir el EOF de la *queue*, por lo que deberá coordinarse con el resto de los word readers para garantizar la propagación de los EOFs hacia los siguientes procesos.

Para la resolución de la tercera consulta, además del campo *abstract*, se tendrá en cuenta el campo *in_language*.

Cada word reader lee el *abstract* de cada artículo y recopila todas las palabras únicas que contiene. Una vez finalizado el procesamiento del artículo, envía las palabras obtenidas al proceso de **sum count** utilizando como clave de hashing el par **{ req_id, word }**.

Cada mensaje enviado al proceso sum count contiene una palabra y dos contadores: *en_count* y *fr_count*, que representan su presencia en artículos en inglés y francés, respectivamente.

Según el valor de *in_language*, el word reader genera uno de los siguientes mensajes:

- Si la palabra está en un artículo en inglés: **[word, en_count=1, fr_count=0]**
- Si está en un artículo en francés: **[word, en_count=1, fr_count=0]**

El objetivo de sum count es acumular los contadores correspondientes a cada palabra, sumando los valores recibidos en los mensajes asociados a una misma palabra y cliente.

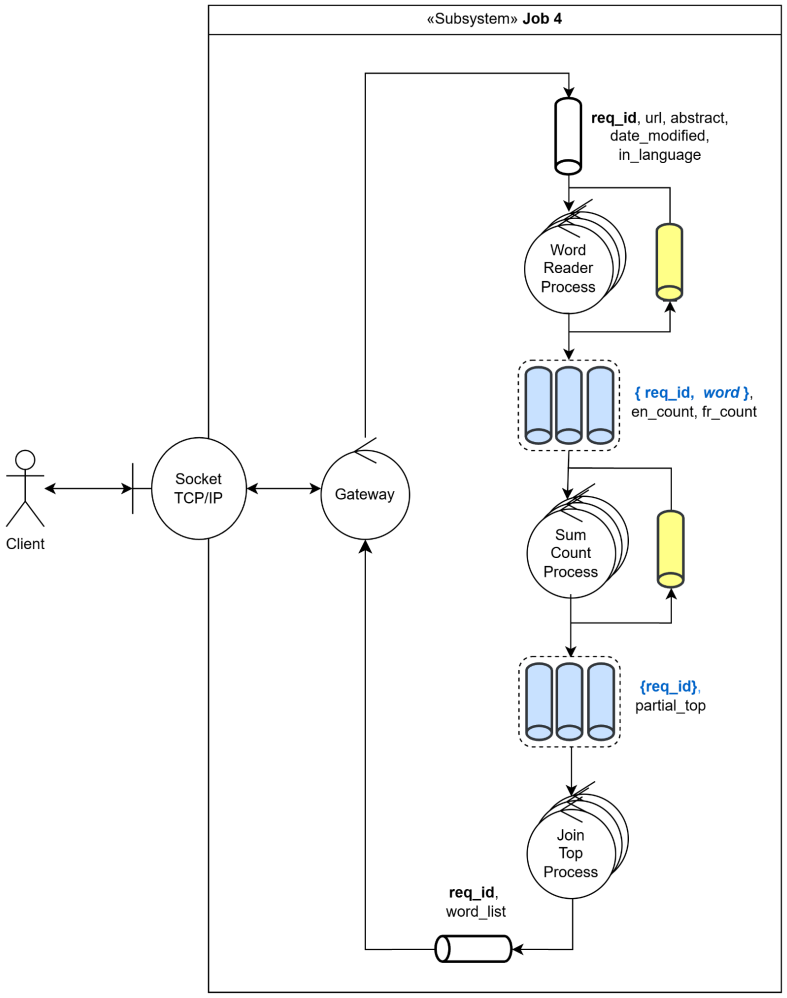
Por ejemplo, si la palabra **ford** aparece en tres artículos en inglés y cuatro artículos en francés correspondientes a una misma solicitud, el proceso acumula los contadores hasta obtener el siguiente resultado: **word: (en_count=3, fr_count=4)**

Una vez que recibido el EOF, se verifican todas las palabras contabilizadas, si en ambos counts son mayores a 0, significa que aparece en artículos de ambos idiomas. Entonces manda aquellas palabras al gateway como resultado.

Solo uno de los procesos sum count será responsable de enviar el EOF final al gateway, por lo que para garantizar que este se envíe en el momento adecuado, los procesos sum count deberán coordinarse entre sí.

Antes de enviar el EOF final, se debe garantizar que todos los procesos sum count hayan recibido y procesado los mensajes enviados por los procesos de word reader y que hayan enviado todos sus resultados al gateway. De esta manera, se evita que el EOF final se envíe antes de que se hayan comunicado todos los resultados correspondientes a la tercera consulta.

2.2.4 Job 4



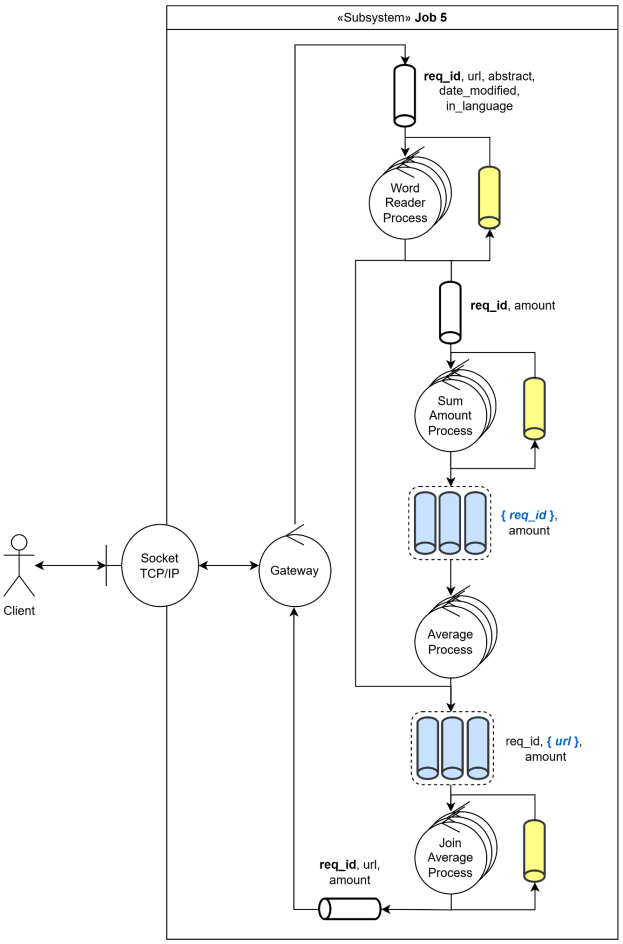
Dado que los artículos procesados pertenecen únicamente a los dos idiomas requeridos para la tercera consulta (inglés y francés), se puede aprovechar el conteo de palabras realizado por el sum count para resolver también la cuarta consulta.

Una vez que el sum count haya terminado de procesar todos los mensajes recibidos, calcula su Top 20 parcial de palabras más frecuentes. Para ello, utiliza los contadores *en_count* y *fr_count*, cuya suma permite obtener el total de apariciones de cada palabra. Este enfoque es válido porque todas las ocurrencias de una misma palabra se envían al mismo sum count. De esta manera, cada proceso dispone del conteo total de cada palabra que le corresponde.

Los Top 20 parciales generados por los distintos procesos sum count se envían a un proceso de join top, que puede escalarse y distribuirse por cliente para procesar las solicitudes de manera independiente.

Una vez que el join top haya recibido todos los Top 20 parciales correspondientes a una solicitud, combina sus resultados y calcula el Top 20 final de palabras más frecuentes. Finalmente, envía el resultado al Gateway.

2.2.4 Job 5



Para la resolución de la quinta consulta, se tendrán en cuenta los campos *url* y *date_modified* de cada artículo, junto con la cantidad de palabras de su *abstract*. El proceso de word reader utiliza el campo *date_modified* para dividir los artículos en dos grupos según su fecha de modificación:

1. Antes de 2021 (*date_modified* < 2021) – Calcula la cantidad de palabras del *abstract* y envía este valor a los procesos de **sum amount** para realizar el cálculo del promedio.
2. En 2021 o posteriormente (*date_modified* >= 2021) – Calcula la cantidad de palabras del *abstract* y envía la *url* del artículo junto con dicha cantidad directamente al proceso de **join average** para su comparación.

El objetivo de los procesos de sum amount es calcular las sumas parciales de las cantidades de palabras de los artículos recibidos; una vez procesados todos los artículos correspondientes, y mediante el mecanismo de coordinación de EOFs, los procesos envían sus sumas parciales al proceso de **average**, el cual recibe las sumas parciales generadas y calcula el promedio de palabras de los artículos modificados antes de 2021.

Una vez obtenido el promedio final, average, lo envía al proceso de join average, seguido de un EOF para indicar que el cálculo ha finalizado.

El proceso de join average recibe, por un lado, las URLs y las cantidades de palabras de los artículos modificados en 2021 o posteriormente, enviados directamente por los word reader, y por otro lado, recibe el promedio calculado por el average.

Una vez que dispone del promedio, compara la cantidad de palabras de cada artículo con dicho valor y envía los resultados correspondientes al gateway.

Finalmente, el join average envía el EOF al gateway una vez que haya recibido los EOFs de ambos flujos de datos: el proveniente de los word reader y el proveniente del average. De esta manera, se garantiza que todos los artículos hayan sido procesados y que los resultados se hayan enviado antes de finalizar el procesamiento de la quinta consulta.

3. Process View

3.1 Procesos concurrentes

Durante la ejecución del sistema existen tres ámbitos principales de concurrencia:

- Las cinco consultas se procesan en paralelo en distintas etapas
- Los componentes pueden tener múltiples instancias para distribuir la carga
- Cada cliente envía artículos y recibe resultados mediante dos hilos concurrentes.

Dentro de cada proceso cliente se utilizan dos hilos de trabajo. El hilo emisor lee y normaliza los artículos, construye los lotes y los envía por TCP y al terminar, transmite el EOF de subida. El hilo receptor permanece activo durante el envío, reconstruye los mensajes recibidos y guarda los resultados según su identificador de consulta, hasta recibir el EOF global del Gateway.

El hilo principal espera la finalización de ambos hilos. Considera exitosa la ejecución cuando se completó el envío, se recibió el EOF global y no se señaló un error ni una interrupción. Entonces renombra los archivos *.partial* a *.csv* y retorna código de salida cero. Si la ejecución falla, retorna código uno y conserva los archivos parciales que se hayan escrito.

Ante SIGTERM, el cliente señala un evento de detención compartido y cierra la conexión TCP para desbloquear las operaciones de red. Los hilos consultan ese evento para interrumpir la lectura y el procesamiento, y se cierran los archivos y recursos abiertos. La interrupción no confirma los resultados como una ejecución completa.

3.2 Query 1

Diagrama de Secuencia

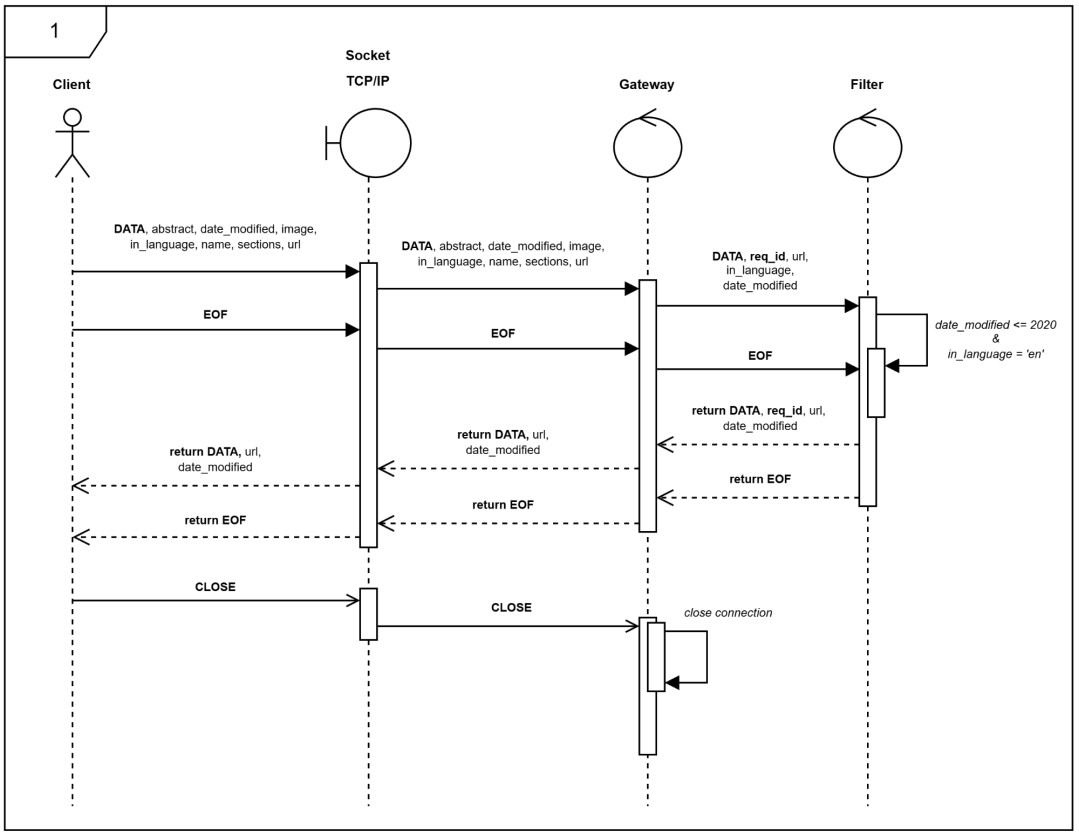
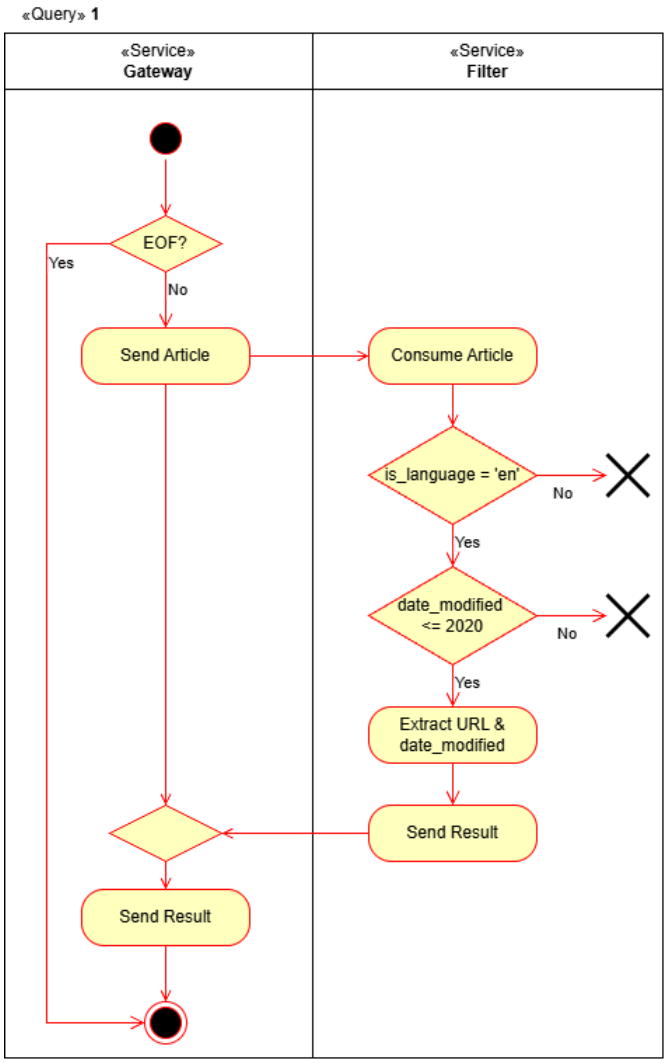


Diagrama de actividad



3.3 Query 2

Diagrama de Secuencia

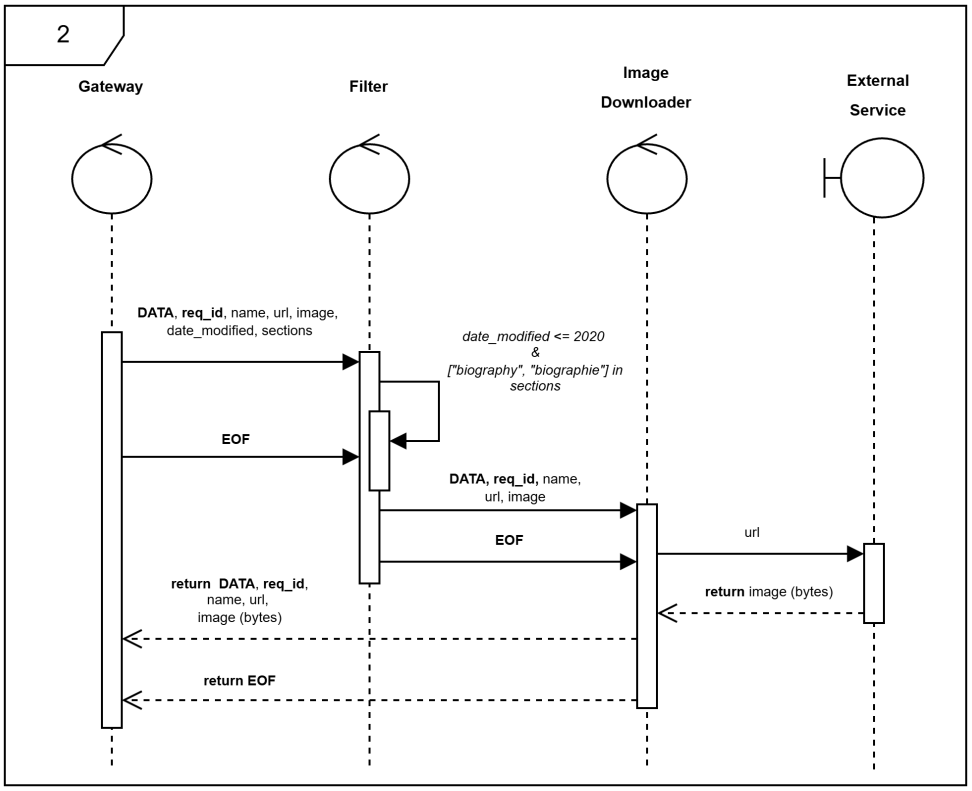
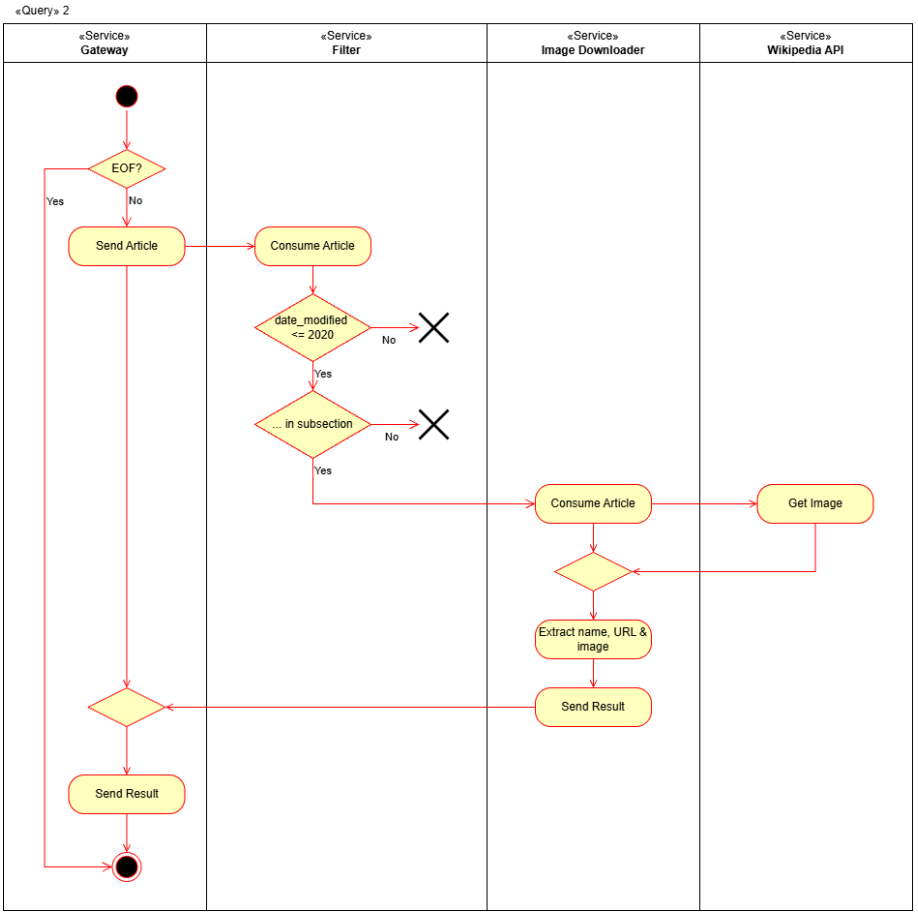
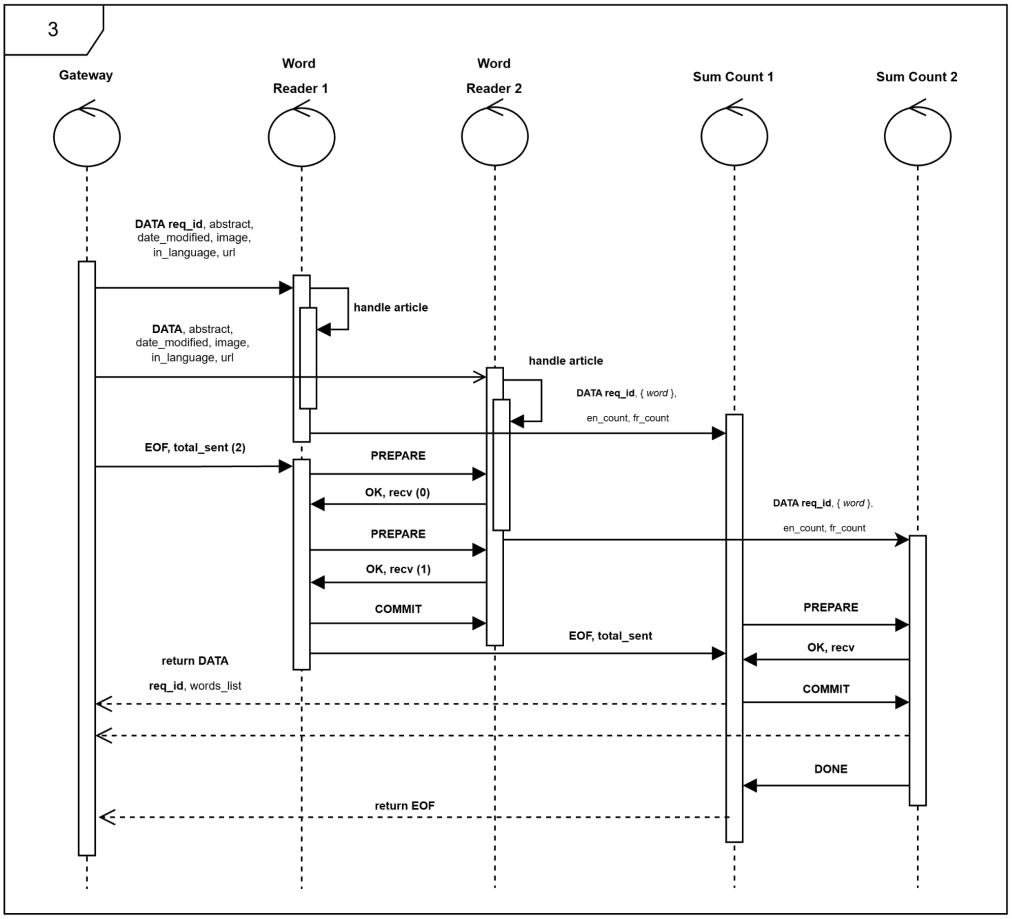


Diagrama de actividad



3.4 Query 3

Diagrama de Secuencia



Este diagrama en particular muestra dos ejemplos del mecanismo de coordinación entre procesos tras la recepción de un EOF. En ambos casos, el proceso que recibe el EOF asume el rol de coordinador. En los ejemplos representados, los coordinadores son **Word Reader 1** y **Sum Count 1**, respectivamente.

El EOF recibido contiene el valor **total_sent**, que representa la cantidad total de mensajes que los procesos del grupo deben haber recibido y procesado en conjunto.

Al recibir el EOF, el coordinador inicia el protocolo enviando un mensaje PREPARE al resto de los procesos involucrados, con el objetivo de verificar si todos han terminado de procesar los mensajes pendientes.

Al recibir un mensaje PREPARE, cada proceso responde con un mensaje OK que incluye el valor recv, correspondiente a la cantidad de mensajes que ese proceso ha recibido.

Una vez que el coordinador recibe las respuestas OK de todos los procesos, suma los valores recv y compara el resultado con total_sent:

- Si la suma de los valores recv coincide con total_sent: se confirma que todos los mensajes enviados han sido recibidos.
- Si ambos valores no coinciden: significa que todavía hay mensajes pendientes de recepción. Por ejemplo, en el caso de Word Reader 2, el proceso responde con recv = 0, lo que indica que aún no ha terminado de recibir y procesar el mensaje. En esta situación, el coordinador espera y vuelve a enviar un mensaje PREPARE para realizar una nueva verificación.

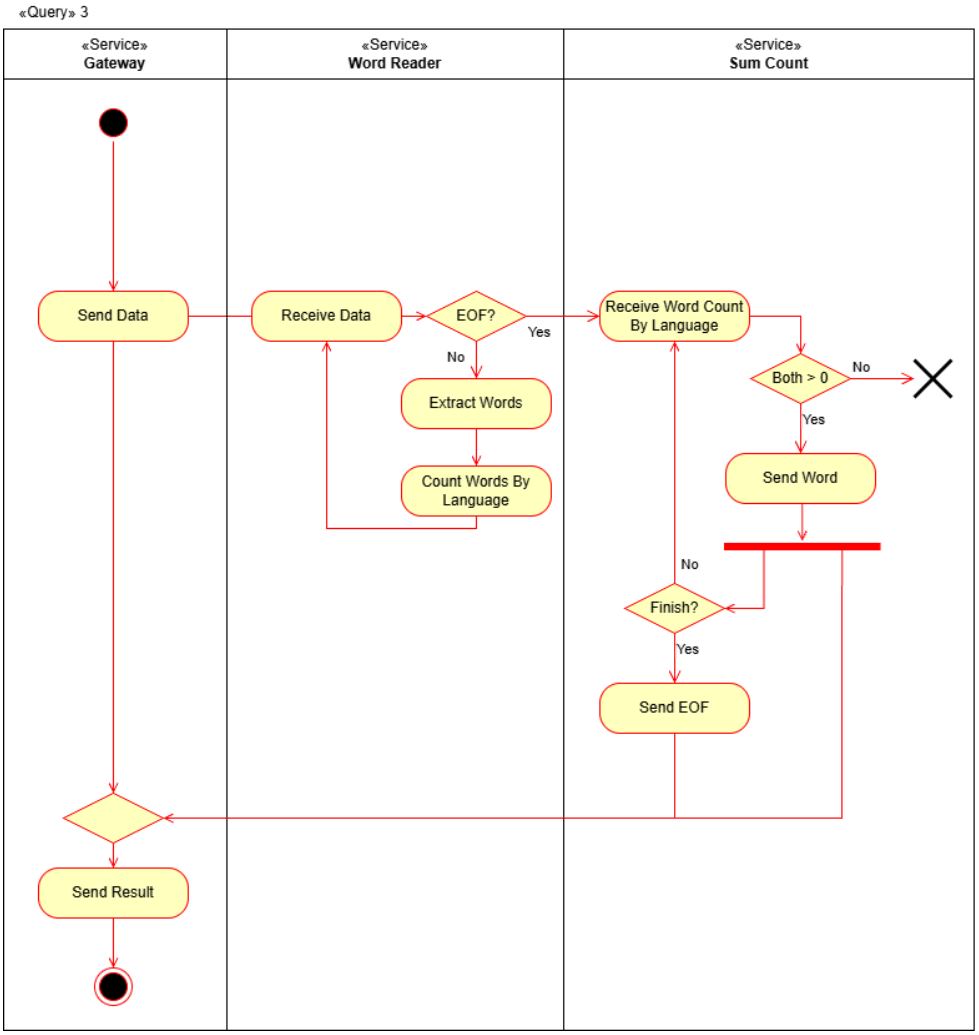
Este procedimiento se repite hasta que se cumpla la condición de que la suma de los mensajes recibidos coincida con total_sent.

Cuando el coordinador verifica que todos los mensajes enviados han sido recibidos, envía un mensaje COMMIT a los procesos involucrados. A continuación, envía el EOF a la siguiente etapa.

En el caso de los procesos Sum Count, el protocolo requiere una etapa adicional de confirmación mediante mensajes DONE, debido a que deben enviar sus resultados finales al Gateway. Cuando los procesos Sum Count reciben el mensaje COMMIT, envían sus resultados finales al Gateway. Una vez que un proceso ha terminado de enviar todos sus resultados, responde con un mensaje DONE al coordinador.

El coordinador espera hasta recibir los mensajes DONE de todos los procesos Sum Count. Entonces envía el EOF final al Gateway, garantizando que todos los resultados hayan sido enviados antes de finalizar el procesamiento.

Diagrama de actividad



3.5 Query 4

Diagrama de Secuencia

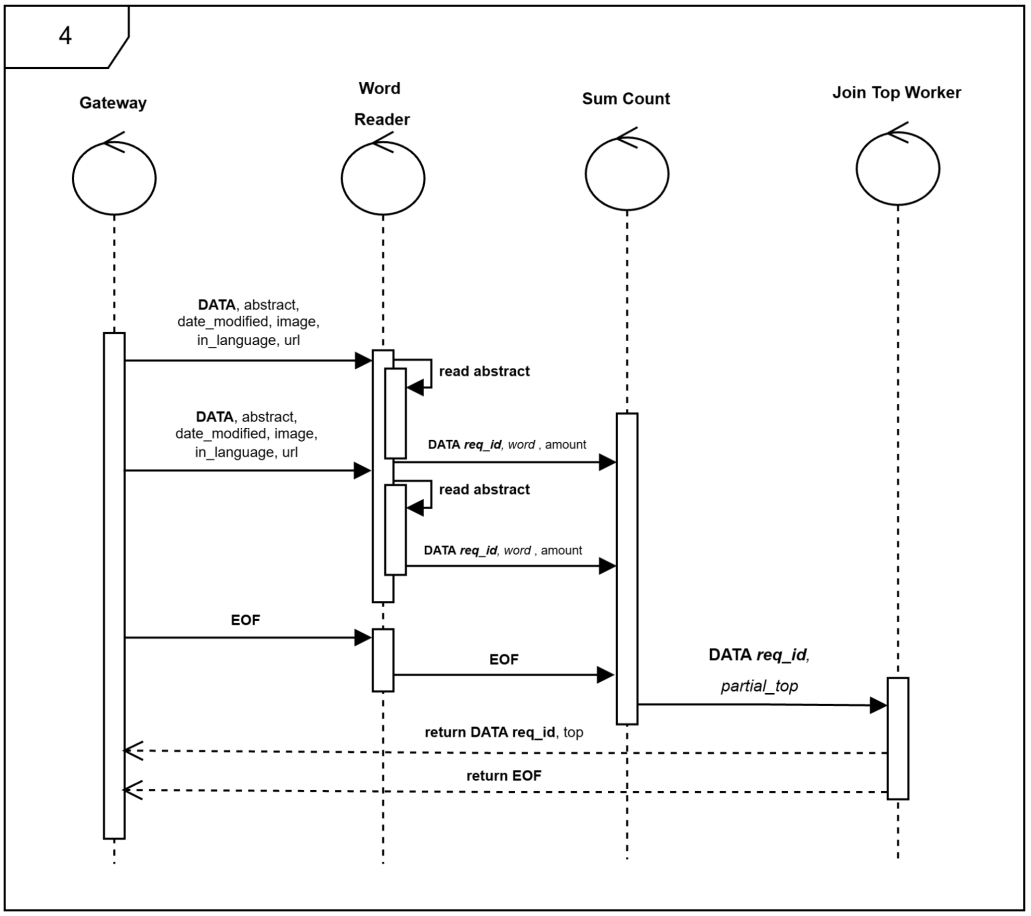
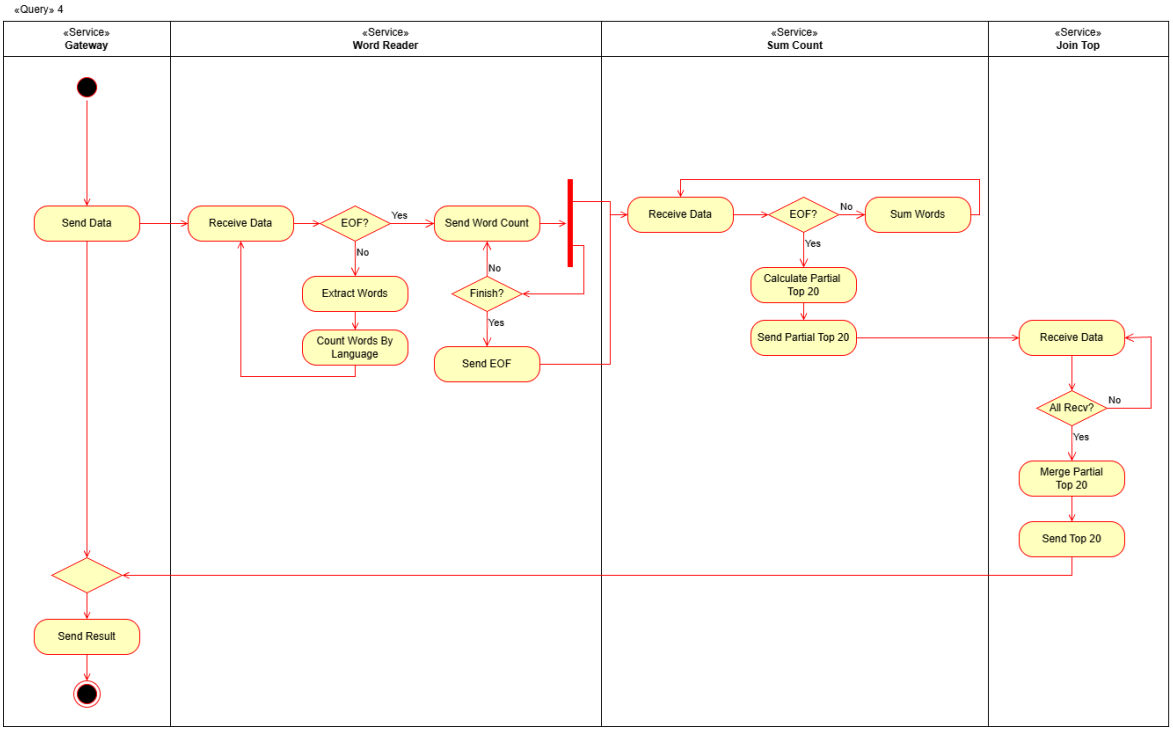


Diagrama de actividad



3.6 Query 5

Diagrama de Secuencia

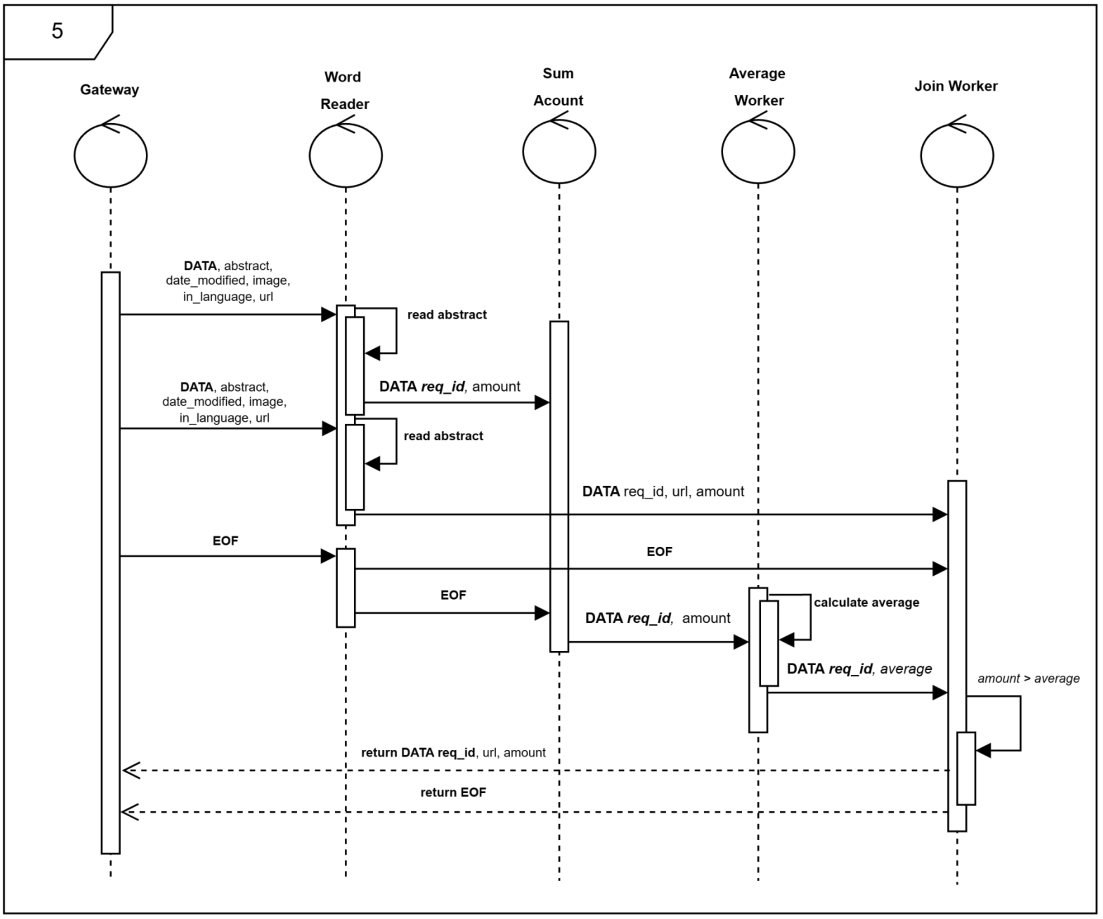
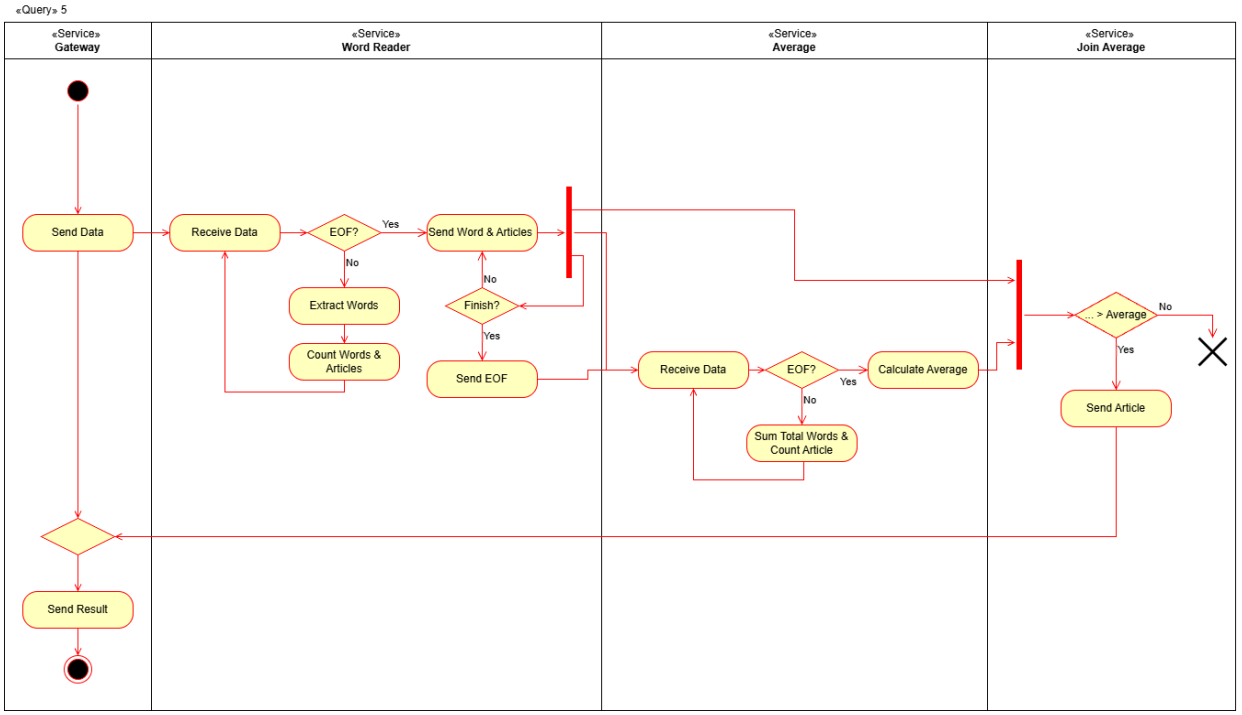


Diagrama de actividad

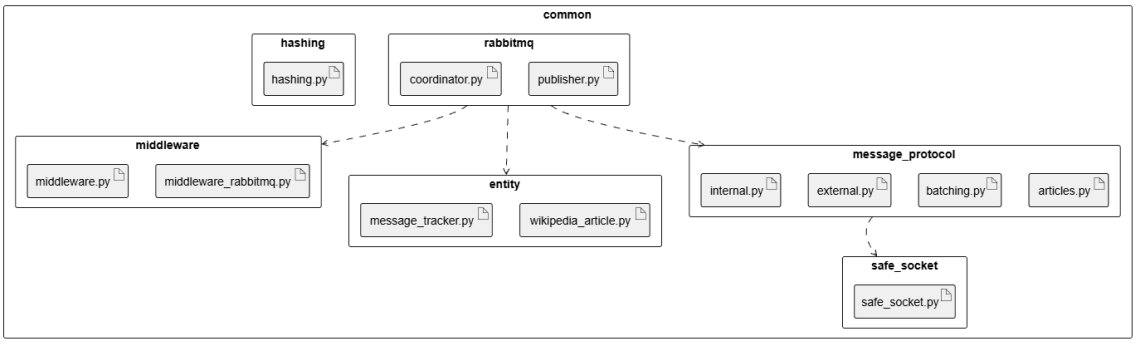


4. Development View

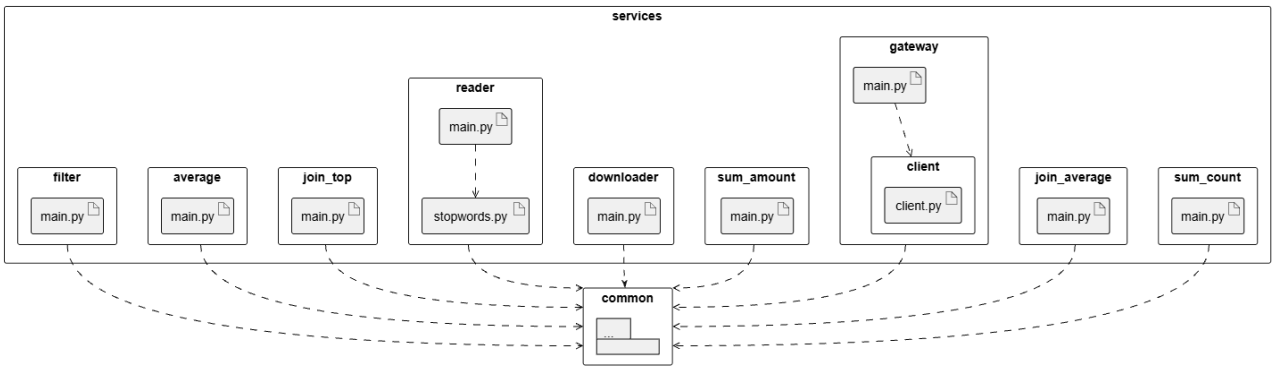
4.1 Diagrama de Paquetes

El **diagrama de paquetes** muestra la organización del sistema desde el punto de vista del código fuente. Cada paquete agrupa componentes con responsabilidades relacionadas, separando la lógica de comunicación, coordinación y procesamiento.

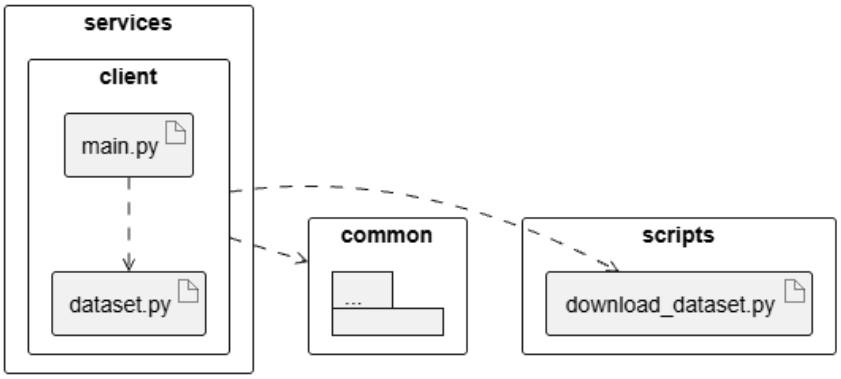
A continuación se presenta el diagrama de paquetes del módulo *common*, el cual agrupa paquetes con funcionalidades que son reutilizados por los demás paquetes del sistema.



A continuación se presenta el diagrama de paquetes del módulo *services*, el cual agrupa los paquetes de procesamiento, es decir, los responsables de resolver las consultas planteadas anteriormente; como se puede observar cada uno de ellos hace uso del módulo *common*.



A continuación se presenta el diagrama de paquetes del módulo *services*, pero para el paquete del cliente, es decir, el programa que deberá ejecutar el cliente para poder realizar el análisis de los artículos de Wikipedia; como podemos observar este hace uso del módulo *common* y *scripts*, que contiene la descarga del dataset de lo articulos de Kaggle.



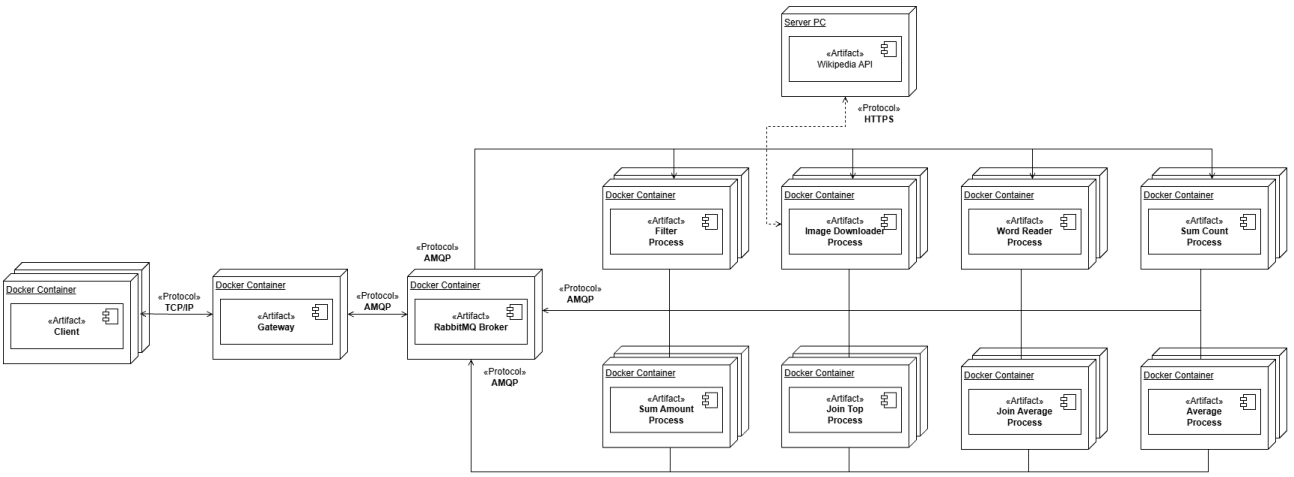
5. Physical View

La arquitectura diseñada se planteó para ser soportada en un entorno de multicomputadoras, donde los distintos componentes pueden ejecutarse en su propia computadora, entonces es necesario que estas computadoras se comuniquen entre sí para cumplir su propósito, por lo que se tomó la decisión de utilizar RabbitMQ como broker para el intercambio de mensajes.

A su vez, cada uno de estos componentes puede escalar de forma horizontal con múltiples instancias, alivianando de esta forma el procesamiento de los componentes que cumplen una misma función.

5.1 Diagrama de Despliegue

El **diagrama de despliegue** muestra la distribución general de los principales componentes del sistema, permitiendo ver de forma visual lo explicado anteriormente.



6. Anexo

6.1 Listado de Tareas

- Implementación del *Client*.
- Lectura del set de datos.
- Implementación del *Gateway*.
- Implementación de Job 1 - *Filter*.
- Implementación de Job 2 - *Image Downloader*.
- Implementación de Job 3: *Word Reader* & *Sum Count*.
- Implementación de Job 4: *Word Reader*, *Sum Count* & *Join Top*.
- Implementación de Job 5: *Word Reader*, *Sum Amount*, *Average* & *Join*.
- Manejo de finalización (*EOF*).
- Implementación de un *graceful quit* ante la recibida de la señal SIGTERM.
- Integración y pruebas del sistema distribuido.

Lautaro Torraca

- Lectura del set de datos.
- Implementación del *Client*.
- Integración y pruebas del sistema distribuido.

Francisco Infanti

- Implementación del *Gateway*.
- Implementación de Job 1 - *Filter*.
- Implementación de Job 2 - *Image Downloader*.
- Manejo de finalización (*EOF*).
- Implementación de un *graceful quit* ante la recibida de la señal SIGTERM.

Franco Alani

- Implementación de Job 3: *Word Reader* & *Sum Count*.
- Implementación de Job 4: *Word Reader*, *Sum Count* & *Join Top*.

- Implementación de Job 5: *Word Reader*, *Sum Amount*, *Average* & *Join*.
- Manejo de finalización (*EOF*).